
A Survey and Technical Review of Lighthouse Attention for Long-Context LLM Pretraining

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Long-context language modeling is increasingly important for document under-
2 standing, repository-level code analysis, agentic reasoning, and retrieval-augmented
3 generation. However, standard scaled dot-product attention scales quadratically
4 with sequence length, making long-context pretraining expensive in both compute
5 and memory. Lighthouse Attention was proposed as a training-time hierarchical
6 selection mechanism that wraps around ordinary dense attention, constructs a
7 multi-resolution representation of queries, keys, and values, selects a compact
8 dense subsequence, and later recovers a dense-attention checkpoint through a short
9 resume phase. This survey reviews Lighthouse Attention from a systems and
10 modeling perspective. We summarize the long-context bottleneck, explain the
11 Lighthouse pipeline, compare it with FlashAttention, block-sparse attention, Native
12 Sparse Attention, MoBA, HISA, and DeepSeek Sparse Attention, and identify
13 open challenges in adaptive budgeting, cross-level coordination, cross-layer per-
14 sistence, uncertainty-aware selection, and reproducible evaluation. The goal is to
15 position Lighthouse Attention within the broader landscape of efficient attention
16 methods and to clarify future research directions for hierarchical selection-based
17 long-context training.

18 1 Introduction

19 Transformer attention remains a dominant bottleneck in long-context language modeling because the
20 standard scaled dot-product attention (SDPA) operator has quadratic time and memory complexity in
21 the sequence length. FlashAttention improves the practical efficiency of exact attention by making
22 the computation IO-aware and reducing high-bandwidth-memory traffic, but it does not remove the
23 underlying quadratic arithmetic scaling of dense attention. As context windows grow from thousands
24 to hundreds of thousands or even millions of tokens, this quadratic cost becomes a central barrier to
25 efficient pretraining.

26 Lighthouse Attention addresses this bottleneck from a training-time perspective. Instead of replacing
27 dense attention permanently, it wraps a selection mechanism around standard SDPA during most
28 of pretraining, then performs a dense-attention resume phase to recover an inference-ready dense
29 checkpoint. The key idea is to construct a hierarchical pyramid over queries, keys, and values, score
30 pooled representations, gather a smaller dense subsequence, run ordinary FlashAttention or SDPA on
31 the selected tokens, and scatter outputs back into the sequence representation.

32 This survey focuses on three questions. First, what design principles make Lighthouse Attention
33 different from earlier sparse or efficient attention methods? Second, what trade-offs arise when
34 selection is hierarchical, gradient-free, and training-only? Third, what research gaps remain after
35 Lighthouse Attention, especially for adaptive selection and uncertainty-aware long-context modeling?

2 Background: The Long-Context Attention Bottleneck

2.1 Dense Attention and Quadratic Scaling

Given query, key, and value matrices $Q, K, V \in \mathbb{R}^{N \times d}$, scaled dot-product attention computes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V. \quad (1)$$

The score matrix QK^T has size $N \times N$, which makes dense attention expensive for large N . This cost affects both training and inference, but long-context pretraining is especially challenging because activations and gradients must also be handled.

2.2 FlashAttention and Exact Kernel Optimization

FlashAttention improves wall-clock performance by tiling the attention computation and reducing memory reads and writes between GPU high-bandwidth memory and on-chip SRAM. It is exact attention, meaning it computes the same mathematical result as standard attention up to numerical considerations. This makes FlashAttention a strong baseline and an important building block for methods such as Lighthouse Attention, which aim to reduce the number of tokens passed into the dense attention kernel rather than replacing the kernel itself.

2.3 Sparse and Selection-Based Attention

Sparse attention methods reduce cost by selecting a subset of key-value positions for each query. These selections may be local windows, fixed patterns, block-level selections, learned indexes, or hierarchical filters. Sparse attention is attractive because it can reduce compute from $O(N^2)$ toward $O(Nk)$, where k is much smaller than N . However, sparse methods must address several practical questions: how to select tokens efficiently, whether selection is differentiable, whether custom kernels are required, whether the method works during training, and whether the final model remains compatible with dense attention.

3 Lighthouse Attention: Method Overview

3.1 Training-Time Wrapper Around Dense Attention

Lighthouse Attention is best understood as a training-time wrapper rather than a permanent inference-time architectural replacement. The method performs selection-based compression before dense attention and decompression after dense attention. Because the selected subsequence is dense and contiguous after gathering, Lighthouse can reuse optimized dense attention kernels such as FlashAttention. This design separates the selection problem from the attention-kernel implementation problem.

3.2 Symmetric Q/K/V Pooling

A central design choice is symmetric pooling: Lighthouse pools queries, keys, and values together across an L -level hierarchy. This differs from many sparse attention methods that keep queries at full resolution while compressing only keys and values. Symmetric pooling makes the hierarchy a multi-scale representation of the entire attention computation rather than only a compressed memory over keys and values.

At level ℓ , a pooled representation can be written abstractly as

$$\bar{X}^{(\ell)} = \text{Pool}_{p^\ell}(X), \quad X \in \{Q, K, V\}, \quad (2)$$

where p is the pooling factor. The result is a pyramid of coarser and finer sequence representations.

3.3 Gradient-Free Scoring and Hierarchical Selection

Lighthouse scores pooled entries using lightweight scorer functions. The public implementation describes scorer variants such as norm, dilated, and GLA-style scoring. The selection is gradient-free,

76 which avoids the need for a complicated differentiable sparse-selection backward kernel. A typical
77 hierarchical selection process starts at a coarse resolution, chooses promising pooled regions, expands
78 those regions to finer children, and repeats until a final token-level selected set is obtained.

79 **3.4 Dense Attention on Selected Tokens**

80 After hierarchical selection, Lighthouse gathers selected entries into a smaller dense subsequence and
81 applies standard dense attention to that subsequence. If the final selected budget is k , the effective
82 attention stage operates over the selected sequence rather than the full context. Conceptually, this
83 changes the dominant attention cost from dense all-to-all attention over N tokens to attention over a
84 selected subset, while preserving compatibility with dense attention kernels.

85 **3.5 Scatter or Decompression**

86 After attention over the selected subsequence, Lighthouse scatters or decompresses outputs back to
87 the original sequence representation. This post-processing step is part of the method’s compression-
88 attention-decompression view. The authors describe Lighthouse as a subquadratic hierarchical pre-
89 and post-processing step around ordinary SDPA.

90 **3.6 Two-Stage Training and Dense-Attention Resume**

91 A key practical feature is the two-stage training recipe. The model is pretrained mostly with
92 Lighthouse Attention, then resumed briefly with standard dense SDPA so that the final checkpoint
93 is dense-attention compatible. This design addresses an important concern for training-time sparse
94 attention: the final model should not become a specialist of its own approximation, but should remain
95 usable as a dense-attention model.

96 **4 Comparison with Related Methods**

97 **4.1 FlashAttention**

98 FlashAttention is an exact attention algorithm that improves attention efficiency through IO-aware
99 tiling and memory-access optimization. It does not select fewer tokens; instead, it computes dense
100 attention more efficiently. Lighthouse is complementary: it reduces the token set before calling a
101 dense attention kernel. In practice, Lighthouse can be viewed as a selection wrapper that benefits
102 from FlashAttention improvements without needing to implement a custom sparse kernel.

103 **4.2 Native Sparse Attention**

104 Native Sparse Attention (NSA) proposes an end-to-end trainable sparse attention mechanism that
105 combines coarse-grained token compression with fine-grained token selection, aiming to preserve
106 global context awareness and local precision while remaining hardware-aligned. Compared with
107 Lighthouse, NSA is a native sparse attention design, whereas Lighthouse emphasizes a training-only
108 wrapper and dense-attention recovery. Both methods use hierarchical ideas, but they differ in whether
109 sparse attention remains part of the main model lifecycle.

110 **4.3 Mixture of Block Attention**

111 MoBA applies a Mixture-of-Experts style routing principle to attention by dividing context into
112 blocks and allowing query tokens to select relevant key-value blocks. MoBA emphasizes block-level
113 routing and the ability to transition between full and sparse modes. Lighthouse differs by constructing
114 a symmetric Q/K/V hierarchy and using hierarchical token selection during pretraining with a later
115 dense resume phase.

116 **4.4 HISA and Hierarchical Indexing**

117 HISA targets the cost of token-level sparse attention indexers by replacing a flat full-prefix scan
118 with a two-stage hierarchical search: coarse block filtering followed by token-level refinement inside

Table 1: High-level comparison of Lighthouse Attention with related efficient attention methods.

Method	Main Category	Core Idea	Strength	Difference from Lighthouse
Exact kernel optimization	IO-aware tiling for exact dense attention	Keeps dense attention mathematically exact	Does not perform hierarchical token selection NSA	FlashAttention Native sparse attention
Combines compression and fine-grained selection	End-to-end sparse training with hardware-aware design	Sparse mechanism is native rather than a removable wrapper MoBA	Block sparse routing	Selects relevant key-value blocks with MoE-like routing
Flexible block-level sparsity	Uses block routing rather than symmetric Q/K/V hierarchical pooling HISA	Hierarchical indexer	Coarse block filtering then token-level refinement	Reduces indexer bottleneck
Improves sparse indexer rather than wrapping dense attention during pretraining DSA	Fine-grained sparse selection	Lightweight indexer selects top key-value entries	Token-level sparse attention	Selection is indexer-driven rather than Lighthouse-style pooled pyramid selection Lighthouse
Training-time hierarchical selection	Symmetric Q/K/V pooling, gradient-free selection, dense-attention resume	Reuses dense kernels and recovers dense checkpoint	Designed as a training-time wrapper around ordinary SDPA	

retained blocks. HISA and Lighthouse both use hierarchy, but their roles differ. HISA improves the indexer for existing sparse attention mechanisms, while Lighthouse uses hierarchy as a training-time compression and decompression wrapper around dense attention.

4.5 DeepSeek Sparse Attention

DeepSeek Sparse Attention (DSA) uses a lightweight indexer and fine-grained token selection for long-context efficiency in DeepSeek sparse-attention variants. The DSA paradigm focuses on selecting key-value entries according to index scores. Lighthouse differs by selecting through a symmetric pooled pyramid and by treating sparse selection as a removable pretraining-time mechanism.

5 Strengths of Lighthouse Attention

Kernel compatibility. Because Lighthouse gathers selected tokens into a dense subsequence, the expensive attention operation can use standard dense kernels. This avoids the engineering burden of maintaining custom sparse attention kernels.

Training-focused design. Many efficient attention methods target inference. Lighthouse is explicitly framed for long-context pretraining and includes a dense-attention recovery phase, making it relevant for training pipelines that ultimately need dense-attention-compatible checkpoints.

Symmetric hierarchy. Pooling Q, K, and V together makes the hierarchy a multi-resolution representation of the attention layer. This symmetry distinguishes Lighthouse from methods that pool only K/V or rely purely on query-to-block routing.

Gradient-free selection. By keeping hierarchical selection gradient-free, Lighthouse avoids a difficult backward-pass design for sparse token selection. This is a practical engineering trade-off:

139 the selection logic is simpler, but the model must rely on the surrounding training dynamics and the
140 dense resume phase to recover final quality.

141 6 Limitations and Open Challenges

142 **Fixed selection budgets.** A fixed top- k budget may be inefficient when attention heads differ in
143 entropy, role, or uncertainty. Some heads may need fewer selected tokens, while others may require a
144 wider beam. This motivates adaptive budget allocation across heads and layers.

145 **Cross-level independence.** Hierarchical selection naturally creates coarse-to-fine dependencies, but
146 the design of how coarse selections guide fine selections remains an open issue. Explicit cross-level
147 priors may reduce redundant or locally misleading selections.

148 **Lack of cross-layer persistence.** Selection is often recomputed independently at each layer. If
149 important tokens recur across depth, maintaining a lightweight cross-layer selection memory could
150 reduce selection variance and improve stability.

151 **Uncertainty-aware fallback.** When selection scores are flat or ambiguous, a fixed top- k rule may
152 select unstable tokens. Confidence-aware fallback mechanisms could widen selection only when
153 uncertainty is high.

154 **Dependence on dense recovery.** The two-stage sparse-to-dense recipe is a strength, but also
155 introduces a dependency: final quality depends on whether the dense resume phase successfully
156 recovers a strong dense-attention model. More studies are needed across model scales, corpora, and
157 context lengths.

158 **Reproducibility and external validation.** As with any new long-context training method, inde-
159 pendent reproductions across hardware, model sizes, tokenizers, and workloads are important. The
160 public implementation helps, but broader validation is needed before conclusions generalize.

161 7 Future Research Directions

162 **Adaptive hierarchical selection.** Future methods can extend Lighthouse by allocating token
163 budgets dynamically across heads, layers, and examples. Entropy, confidence, or gradient-free
164 uncertainty statistics could guide such allocation.

165 **Cross-level guidance.** Coarse pooled scores can be used not only as filters but also as priors for
166 fine-level selection. Learned coarse-to-fine guidance may reduce redundant selection and improve
167 routing quality.

168 **Cross-layer selection memory.** Lightweight state variables could track which tokens or regions
169 remain important across depth. Such memory may improve consistency without introducing a full
170 recurrent hidden state.

171 **Hybrid sparse-dense training schedules.** The sparse-to-dense recovery phase is central to Light-
172 house. Future work can explore curriculum schedules that interleave sparse and dense attention or
173 gradually anneal selection budgets.

174 **Evaluation beyond loss and throughput.** Long-context methods should be evaluated on retrieval,
175 reasoning over long documents, codebase-level understanding, memory stability, and robustness
176 under adversarial or distractor-heavy contexts. Reporting only throughput is insufficient because
177 selection quality directly affects learned representations.

178 8 Positioning PHAROS Relative to Lighthouse

179 Lighthouse Attention establishes a strong baseline for training-time hierarchical selection, but its
180 design leaves room for adaptive and uncertainty-aware extensions. PHAROS can be viewed as one

181 such extension direction. Depth-Sounding Budget Allocation addresses fixed per-head budgets; Range
182 Light Cross-Level Guidance addresses the lack of explicit coarse-to-fine guidance; Tidal Momentum
183 Selection addresses stateless selection across transformer depth; and Fog Gate Confidence Fallback
184 addresses low-confidence selection. This positioning does not require unsupported numerical claims:
185 it follows directly from the structural gaps identified in Lighthouse-style hierarchical selection.

186 **9 Conclusion**

187 Lighthouse Attention is an important contribution to long-context LLM pretraining because it reframes
188 sparse selection as a removable training-time wrapper around dense attention rather than a permanent
189 replacement for the attention operator. Its combination of symmetric Q/K/V pooling, gradient-free
190 hierarchical selection, dense subsequence attention, and dense-attention recovery creates a practical
191 pathway for reducing pretraining cost while preserving compatibility with ordinary attention models.
192 Compared with FlashAttention, NSA, MoBA, HISA, and DSA, Lighthouse occupies a distinct point
193 in the design space: it is hierarchical, training-focused, kernel-compatible, and recoverable to dense
194 attention. Open problems remain in adaptive budgeting, cross-level guidance, cross-layer memory,
195 uncertainty-aware fallback, and reproducible evaluation. These gaps define promising directions for
196 future research in efficient long-context attention.

197 **References**